General Disclosure

I will be using Chat GPT, specifically GPT-5.6 Sol

Verification Statement

I am verifying that I won't blindly copy code without reading and understanding the code
The code will be tested and work as intended
I will acknowledge any bugs I find

Prompt 1:

I am writing a C++ program for a programming assignment involving manual parsing of IPv4 addresses from noisy text.

Please help me implement the following function:

bool extractIPv4(const std::string& str, unsigned long& outAddress, int& outPort);

The program must scan arbitrary text looking for a valid IPv4 address, optionally followed by a port.

Important parsing rules:

- Only digits, periods, and colons can belong to a candidate token.
- Any other character separates candidate tokens and should otherwise be ignored.
- A candidate must be validated in its entirety. Do not accept a valid IPv4 substring from inside a larger invalid sequence of digits, periods, or colons.
- An IPv4 address contains exactly four octets separated by three periods.
- Each octet contains 1–3 digits.
- Each octet must have a numeric value from 0 through 255.
- Leading zeros are forbidden except for the value 0 itself. For example, 0 is valid, while 00, 01, and 001 are invalid.
- After the fourth octet, an optional colon and port may appear.
- A port contains 1–5 digits and must have a value from 0 through 65535.
- Ports use the same leading-zero restriction.
- If a colon is present but the port is malformed, reject the entire candidate, including the otherwise-valid IPv4 address.
- Extra periods or colons directly adjacent to an otherwise valid address make that entire candidate invalid.
- Digit values must be accumulated manually character by character.

Do not use:
atoi, atol, atoll, strtol, strtoul, strtod, stoi, stol, stoul, sscanf, scanf numeric conversion, regex, inet_aton, inet_pton, inet_addr, or similar parsing/conversion functions.

On success:

- return true
- store the 32-bit IPv4 value in outAddress
- store the port in outPort, or -1 if no port exists

The IPv4 integer should be calculated as:

(octet1 << 24) | (octet2 << 16) | (octet3 << 8) | octet4

On failure:

- return false
- set outAddress to 0
- set outPort to -1

Please write readable beginner/intermediate-level C++ rather than an overly clever solution. Break the parsing into understandable steps and comment the important validation logic.

After producing the code, explain the algorithm and identify edge cases that I should test rather than simply claiming the implementation is correct.

Code

```
#include <string>

// Returns true if c is allowed to be part of an IPv4 candidate token.
bool isCandidateChar(char c)
{
    return (c >= '0' && c <= '9') || c == '.' || c == ':';
}


// Parses a decimal number starting at index.
//
// Rules enforced here:
// - Must contain at least one digit.
// - Cannot contain more than maxDigits digits.
// - Leading zeros are forbidden unless the number is exactly "0".
// - Value cannot exceed maxValue.
// - Value is accumulated manually, digit by digit.
```

```
//
// On success, index is left pointing at the first non-digit character.
bool parseNumber(const std::string& token,
                 std::size_t& index,
                 int maxDigits,
                 unsigned int maxValue,
                 unsigned int& value)
{
    if (index >= token.size() ||
        token[index] < '0' || token[index] > '9')
    {
        return false;
    }

    std::size_t start = index;
    value = 0;
    int digitCount = 0;

    while (index < token.size() &&
           token[index] >= '0' && token[index] <= '9')
    {
        // Too many digits.
        if (digitCount >= maxDigits)
        {
            return false;
        }

        unsigned int digit = token[index] - '0';

        // Manual decimal accumulation.
        value = value * 10 + digit;

        ++digitCount;
        ++index;
    }

    // Leading zero is only legal for the number 0 itself.
    // Examples:
    // "0"   -> valid
    // "00"  -> invalid
    // "01"  -> invalid
    if (digitCount > 1 && token[start] == '0')
    {
        return false;
```

```
    }

    if (value > maxValue)
    {
        return false;
    }

    return true;
}


// Validates one complete candidate token.
bool parseIPv4Candidate(const std::string& token,
                        unsigned long& address,
                        int& port)
{
    std::size_t index = 0;
    unsigned int octets[4];

    // Parse exactly four octets.
    for (int i = 0; i < 4; ++i)
    {
        if (!parseNumber(token, index, 3, 255, octets[i]))
        {
            return false;
        }

        // The first three octets must be followed by a period.
        if (i < 3)
        {
            if (index >= token.size() || token[index] != '.')
            {
                return false;
            }

            ++index; // Skip the period.
        }
    }

    // If we are at the end, there is no port.
    if (index == token.size())
    {
        port = -1;
    }
```

```
    else
    {
        // Anything after the IPv4 address must begin with exactly one colon.
        if (token[index] != ':')
        {
            return false;
        }

        ++index; // Skip the colon.

        unsigned int parsedPort;

        if (!parseNumber(token, index, 5, 65535, parsedPort))
        {
            return false;
        }

        // The port must consume the rest of the candidate.
        //
        // Rejects things like:
        // 192.168.1.1:80:90
        // 192.168.1.1:80.
        // 192.168.1.1:80:
        if (index != token.size())
        {
            return false;
        }

        port = static_cast<int>(parsedPort);
    }

    // Build the 32-bit IPv4 value.
    address =
        (static_cast<unsigned long>(octets[0]) << 24) |
        (static_cast<unsigned long>(octets[1]) << 16) |
        (static_cast<unsigned long>(octets[2]) << 8)  |
        static_cast<unsigned long>(octets[3]);

    return true;
}


bool extractIPv4(const std::string& str,
                 unsigned long& outAddress,
```

```
            int& outPort)
{
    // Required failure/default values.
    outAddress = 0;
    outPort = -1;

    std::size_t index = 0;

    while (index < str.size())
    {
        // Ignore separators until we find a possible candidate.
        while (index < str.size() && !isCandidateChar(str[index]))
        {
            ++index;
        }

        if (index >= str.size())
        {
            break;
        }

        // Find the ENTIRE candidate token.
        //
        // This is important because we are not allowed to extract a valid
        // address from the middle of a larger invalid sequence.
        std::size_t start = index;

        while (index < str.size() && isCandidateChar(str[index]))
        {
            ++index;
        }

        std::string candidate = str.substr(start, index - start);

        unsigned long address;
        int port;

        if (parseIPv4Candidate(candidate, address, port))
        {
            outAddress = address;
            outPort = port;
            return true;
        }
```

```
        // If this entire candidate was invalid, keep searching after it.
    }

    // No valid candidate was found.
    outAddress = 0;
    outPort = -1;
    return false;
}
```

Sample Text

```
connecting to 192.168.1.1 now
server=10.0.0.255:8080end
192a168.1.1.1
192.168.1.1.
Connection from 192.168.1.1 refused
192.168.01.1
1.2.3.4:99999
12.34.56
no number here
0.0.0.0
255.255.255.255
1.2.3.4
1.2.3.4:0
1.2.3.4:65535
256.1.1.1
1.256.1.1
1.1.256.1
1.1.1.256
999.999.999.999
01.2.3.4
1.02.3.4
1.2.03.4
1.2.3.04
00.0.0.0
1.2.3.4:00
1.2.3.4:080
1.2.3.4:00001
.1.2.3.4
1.2.3.4.
1..2.3.4
1.2..3.4
1.2.3..4
1.2.3
```

1.2.3.4.5
1.2.3.4:
1.2.3.4::80
1.2.3.4:80:90
1.2.3.4:80.
:1.2.3.4
:::
...
abc1.2.3.4xyz
abc1.2.3.4:80xyz
999a192.168.1.1
999.999.999.999garbage192.168.1.1
192.168.1.1.999 garbage 10.0.0.1
1.2.3.4x5.6.7.8
123a1.2.3.4b456
garbage 10.20.30.40 more garbage
999.999.999.999 and 172.16.0.1
1.2.3 invalid then 8.8.8.8
1.2.3.4 and 5.6.7.8
10.0.0.1:1
10.0.0.1:9
10.0.0.1:10
10.0.0.1:9999
10.0.0.1:65534
10.0.0.1:65535
10.0.0.1:65536
10.0.0.1:99999
10.0.0.1:100000
1234.1.1.1
1.1234.1.1
1.1.1234.1
1.1.1.1234
1111.2222.3333.4444
1234567890.1.2.3
1.2.3.4:123456
hello!!!8.8.8.8???
IP=[127.0.0.1]
address=>192.0.2.1:443;
noise 300.1.1.1 noise 203.0.113.25:8080 done
x1.1.1.1y
before:::192.168.1.1 after
before...192.168.1.1 after

END

Outputs from run

Cannot run as there was no main function or way to test output

Prompt 2:

Edit the code and add a main function that will read from a text file input from the user at run time. The function should prompt the user to input the name of a file that is in it's root folder, the input should be treated as a block of text and be able to iterate through it checking for IPv4 as stated. The program should end and wait for user input either a y/Y to continue with another text file to run or n/N where it'll close the program

Updated Code:

```
int main()
{
   char choice = 'Y';

   // Continue running until the user chooses N or n.
   while (choice == 'Y' || choice == 'y')
   {
      std::string fileName;

      std::cout << "Enter the name of the text file: ";
      std::getline(std::cin, fileName);

      // Attempt to open the file.
      std::ifstream inputFile(fileName);

      if (!inputFile.is_open())
      {
         std::cout << "\nError: Could not open file \""
               << fileName << "\".\n";
         std::cout << "Make sure the file is in the program's "
               << "working directory.\n";
      }
      else
      {
         // Read the entire file into one string.
         std::stringstream buffer;
         buffer << inputFile.rdbuf();

         std::string text = buffer.str();
```

```
    inputFile.close();

    unsigned long address = 0;
    int port = -1;

    std::cout << "\nSearching file...\n\n";

    if (extractIPv4(text, address, port))
    {
        std::cout << "Valid IPv4 address found: ";

        printIPv4(address);

        std::cout << "\n";

        std::cout << "32-bit address value: "
                  << address << "\n";

        if (port != -1)
        {
            std::cout << "Port: "
                      << port << "\n";
        }
        else
        {
            std::cout << "Port: None\n";
        }
    }
    else
    {
        std::cout << "No valid IPv4 address was found.\n";
    }
}

// Ask whether another file should be checked.
while (true)
{
    std::cout << "\nWould you like to check another file? "
              << "(Y/N): ";

    std::string answer;
    std::getline(std::cin, answer);

    if (answer.length() == 1 &&
```

```
            (answer[0] == 'Y' || answer[0] == 'y'))
        {
            choice = answer[0];
            std::cout << "\n";
            break;
        }

        if (answer.length() == 1 &&
            (answer[0] == 'N' || answer[0] == 'n'))
        {
            choice = answer[0];
            break;
        }

        std::cout << "Invalid input. Please enter Y or N.\n";
    }
  }

  std::cout << "\nProgram closed.\n";

  return 0;
}
```

Sample Text

Same as prompt 1

Outputs from run

Searching file...

Valid IPv4 address found: 192.168.1.1
32-bit address value: 3232235777
Port: None

Would you like to check another file? (Y/N): n

Program closed.

Prompt 3

I misread a requirement of this assignment. Rewrite our main function to take input from the user one line at a time then tell the user if that input is valid or not. Then ask the user if they wish to do another line.

Issue

The current set up reads an entire text file and outputs the first available correct address, which is not what we want. We need to test side by side.

Code changes

```
int main()
{
    std::string input;

    while (true)
    {
        std::cout << "Enter a string (or 'END' to quit): ";
        std::getline(std::cin, input);

        // END is case-sensitive.
        if (input == "END")
        {
            std::cout << "Program terminated." << std::endl;
            break;
        }

        unsigned long address = 0;
        int port = -1;

        if (extractIPv4(input, address, port))
        {
            // Extract the four octets from the packed address.
            unsigned long a = (address >> 24) & 255;
            unsigned long b = (address >> 16) & 255;
            unsigned long c = (address >> 8) & 255;
            unsigned long d = address & 255;

            std::cout << "Extracted IPv4 address: "
                      << a << "."
                      << b << "."
                      << c << "."
```

```
                << d
                << " (decimal value: "
                << address
                << ", port: ";

            if (port == -1)
            {
                std::cout << "none";
            }
            else
            {
                std::cout << port;
            }

            std::cout << ")" << std::endl;
        }
        else
        {
            std::cout << "Invalid input: no valid IPv4 address found"
                << std::endl;
        }
    }

    return 0;
}
```

Sample text

Same as above but now I’m entering it one line at a time

Outputs from run

```
PS D:\Github Repos\AI-Assited-IPv4-Extraction> g++ noisytextidentifier.cpp -o
noisytextidentifier.exe
PS D:\Github Repos\AI-Assited-IPv4-Extraction> .\noisytextidentifier.exe
Enter a string (or 'END' to quit): connecting to 192.168.1.1 now
Extracted IPv4 address: 192.168.1.1 (decimal value: 3232235777, port: none)
Enter a string (or 'END' to quit): server=10.0.0.255:8080end
Extracted IPv4 address: 10.0.0.255 (decimal value: 167772415, port: 8080)
Enter a string (or 'END' to quit): 192a168.1.1.1
Extracted IPv4 address: 168.1.1.1 (decimal value: 2818638081, port: none)
Enter a string (or 'END' to quit): 192.168.1.1.
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): Connection from 192.168.1.1 refused
```

Extracted IPv4 address: 192.168.1.1 (decimal value: 3232235777, port: none)
Enter a string (or 'END' to quit): 192.168.01.1
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.2.3.4:99999
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 12.34.56
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): no number here
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 0.0.0.0
Extracted IPv4 address: 0.0.0.0 (decimal value: 0, port: none)
Enter a string (or 'END' to quit): 255.255.255.255
Extracted IPv4 address: 255.255.255.255 (decimal value: 4294967295, port: none)
Enter a string (or 'END' to quit): 1.2.3.4
Extracted IPv4 address: 1.2.3.4 (decimal value: 16909060, port: none)
Enter a string (or 'END' to quit): 1.2.3.4:0
Extracted IPv4 address: 1.2.3.4 (decimal value: 16909060, port: 0)
Enter a string (or 'END' to quit): 1.2.3.4:65535
Extracted IPv4 address: 1.2.3.4 (decimal value: 16909060, port: 65535)
Enter a string (or 'END' to quit): 256.1.1.1
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.256.1.1
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.1.256.1
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.1.1.256
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 999.999.999.999
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 01.2.3.4
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.02.3.4
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.2.03.4
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.2.3.04
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 00.0.0.0
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.2.3.4:00
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.2.3.4:080
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.2.3.4:00001

Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): .1.2.3.4
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.2.3.4.
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1..2.3.4
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.2..3.4
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.2.3..4
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.2.3
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.2.3.4.5
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.2.3.4:
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.2.3.4::80
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.2.3.4:80:90
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.2.3.4:80.
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): :1.2.3.4
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): :::
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): ...
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): abc1.2.3.4xyz
Extracted IPv4 address: 1.2.3.4 (decimal value: 16909060, port: none)
Enter a string (or 'END' to quit): abc1.2.3.4:80xyz
Extracted IPv4 address: 1.2.3.4 (decimal value: 16909060, port: 80)
Enter a string (or 'END' to quit): 999a192.168.1.1
Extracted IPv4 address: 192.168.1.1 (decimal value: 3232235777, port: none)
Enter a string (or 'END' to quit): 999.999.999.999garbage192.168.1.1
Extracted IPv4 address: 192.168.1.1 (decimal value: 3232235777, port: none)
Enter a string (or 'END' to quit): 192.168.1.1.999 garbage 10.0.0.1
Extracted IPv4 address: 10.0.0.1 (decimal value: 167772161, port: none)
Enter a string (or 'END' to quit): 1.2.3.4x5.6.7.8
Extracted IPv4 address: 1.2.3.4 (decimal value: 16909060, port: none)
Enter a string (or 'END' to quit): 123a1.2.3.4b456
Extracted IPv4 address: 1.2.3.4 (decimal value: 16909060, port: none)
Enter a string (or 'END' to quit): garbage 10.20.30.40 more garbage

Extracted IPv4 address: 10.20.30.40 (decimal value: 169090600, port: none)
Enter a string (or 'END' to quit): 999.999.999.999 and 172.16.0.1
Extracted IPv4 address: 172.16.0.1 (decimal value: 2886729729, port: none)
Enter a string (or 'END' to quit): 1.2.3 invalid then 8.8.8.8
Extracted IPv4 address: 8.8.8.8 (decimal value: 134744072, port: none)
Enter a string (or 'END' to quit): 1.2.3.4 and 5.6.7.8
Extracted IPv4 address: 1.2.3.4 (decimal value: 16909060, port: none)
Enter a string (or 'END' to quit): 10.0.0.1:1
Extracted IPv4 address: 10.0.0.1 (decimal value: 167772161, port: 1)
Enter a string (or 'END' to quit): 10.0.0.1:9
Extracted IPv4 address: 10.0.0.1 (decimal value: 167772161, port: 9)
Enter a string (or 'END' to quit): 10.0.0.1:10
Extracted IPv4 address: 10.0.0.1 (decimal value: 167772161, port: 10)
Enter a string (or 'END' to quit): 10.0.0.1:9999
Extracted IPv4 address: 10.0.0.1 (decimal value: 167772161, port: 9999)
Enter a string (or 'END' to quit): 10.0.0.1:65534
Extracted IPv4 address: 10.0.0.1 (decimal value: 167772161, port: 65534)
Enter a string (or 'END' to quit): 10.0.0.1:65535
Extracted IPv4 address: 10.0.0.1 (decimal value: 167772161, port: 65535)
Enter a string (or 'END' to quit): 10.0.0.1:65536
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 10.0.0.1:99999
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 10.0.0.1:100000
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1234.1.1.1
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.1234.1.1
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.1.1234.1
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.1.1.1234
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1111.2222.3333.4444
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1234567890.1.2.3
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.2.3.4:123456
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): hello!!!8.8.8.8???
Extracted IPv4 address: 8.8.8.8 (decimal value: 134744072, port: none)
Enter a string (or 'END' to quit): IP=[127.0.0.1]
Extracted IPv4 address: 127.0.0.1 (decimal value: 2130706433, port: none)
Enter a string (or 'END' to quit): address=>192.0.2.1:443;

```
Extracted IPv4 address: 192.0.2.1 (decimal value: 3221225985, port: 443)
Enter a string (or 'END' to quit): noise 300.1.1.1 noise 203.0.113.25:8080 done
Extracted IPv4 address: 203.0.113.25 (decimal value: 3405803801, port: 8080)
Enter a string (or 'END' to quit): x1.1.1.1y
Extracted IPv4 address: 1.1.1.1 (decimal value: 16843009, port: none)
Enter a string (or 'END' to quit): before:::192.168.1.1 after
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): before...192.168.1.1 after
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit):
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): END
Program terminated.
```